VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM
HORVÁTH BOLDIZSÁR
KÖZGAZDASÁGI ÉS INFORMATIKAI
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

Jogsifoglaló alkalmazás dokumentációja

Készítették:

CSABA BENCE

TAKÁCS BARNABÁS

SZOMBATHELY

2026

Tartalom

1. Bevezetés 1

1.1 A Jogsifoglaló alkalmazás bemutatása 1

1.2 Témaválasztás indoklása és piaci igény 1

1.3 Célközönség 1

2. Fejlesztői dokumentáció 2

2.1 Fejlesztői környezet 2

2.2 Műszaki feltételek 2

3. Csapatmunka 3

3.1 Feladatok elosztása 3

3.2 Kommunikáció 3

4. Kialakított adatbázis és részletes bemutatása 4

4.1 Az adatbázis ER modell leírása 4

4.2 Egyedtípusok 4

4.3 Az adatbázis kapcsolatok és kényszerfeltételek 8

5. Backend felépítése és projektstruktúra (C# / .NET Core Web API) 11

5.1 A rétegek feladatai 11

5.2 A kontrollerek részletes bemutatása 14

5.3 Biztonság és Jogosultságkezelés (Authorization & Authentication) 17

6. Frontend megvalósítása (React & Vite) 18

6.1 React és a Vite 18

6.2 Alkalmazott külső könyvtárak és függőségek 18

6.3 Komponens alapú felépítés 19

6.4 Állapotkezelés és Kliensoldali Útválasztás 19

6.5 Autentikációs és Validációs Modulok 20

6.6 Reszponzív Navigációs Rendszer 20
6.7 Naptár és Foglalási Logika 21
6.8 Időpont-generátor Modul 21
6.9 Áttekintő és Vizualizációs Panelek 22
6.10 Adminisztrációs Rendszer 23
6.11 Stíluslapok és Reszponzív Kialakítás (CSS) 24
7. Tesztelés 25
7.1 Backend egységtesztek (Unit Tests) 25
7.2 Forgatókönyv alapú tesztelés (Scenario) 26
7.3 Tesztek futtatása és eredmények 27
8. Telepítési útmutató 28
8.1 Adatbázis konfigurálása (XAMPP) 28
8.2 Backend (C# .NET API) indítása 28
8.3 Frontend (React) indítása 28
9. Nehézségek és megoldások 29
10. Továbbfejlesztési lehetőségek 30
11. Összegzés 31
Mellékletek 32
ER-Modell 32
Regisztrációs oldal 33

1. Bevezetés

1.1 A Jogsifoglaló alkalmazás bemutatása

A Jogsifoglaló egy modern, reszponzív webes alkalmazás, melynek célja az autósiskolák mindennapi időpontkezelés és adminisztrációját segíti. Célja, hogy egyszerűbbé tegye a kapcsolattartást az oktatók és a tanulók között, valamint leváltsa a sokszor körülményes, manuális időpont-egyeztetési módszereket.

Az alkalmazás központi eleme egy naptáralapú foglalási rendszer. Ennek segítségével az oktatók könnyedén megadhatják szabad időpontjaikat, illetve kezelhetik saját beosztásukat. A tanulók egy átlátható felületen valós időben láthatják az elérhető időpontokat, majd gyorsan és egyszerűen lefoglalhatják a számukra megfelelő vezetési órát.

1.2 Témaválasztás indoklása és piaci igény

A projekt ötlete a vezetési órák során tapasztalt problémákból született. A mai világban az oktatók telefonon, oktatás vagy vezetés közben egyeztették az időpontokat, ami több nehézséget okozhat, emellett baleset veszélyes is. Felmerülhetnek közlekedési kockázatok, mint az előbb említett vezetés közbeni telefonálás. Adminisztrációs problémák, papíralapú naptár használata esetén, felmerülhetnek elírások, félreértések vagy akár ütköznek az időpontok. A tanulók számára is nehézkes a tervezés, egy ilyen alkalmazással pár kattintás után látnak a lefoglalt időpontjaikat.

1.3 Célközönség

Tanulók: Könnyedén tudnak időpontot foglalni vezetési órákra és egy helyen láthatják az időpontjaikat mikor vannak óráik. Rugalmasabb és gyorsabb az ügyintézés, ami a mai modern világban az egyik legfontosabb dolog.

Oktatók: Nem kell vezetés közben a telefonálni és a naptárba írogatni, ami balesetveszélyes is. Csak létrehozzák a szabad időpontokat az alkalmazásba, és kész. Bármikor elérhetik a telefonjukon, hogy ki, mikor jön hozzájuk, így nincs több elfelejtett óra vagy egymásra szervezés.

Autósiskolák: Az iskola látja az összes oktatót és tanulót egy helyen. Könnyebb az adminisztráció, mert nem papíron történik, hanem minden digitális. Látják a befizetéseket, a levezetett órákat, és az egész iskola modernebbnek tűnik a tanulók szemében.

2. Fejlesztői dokumentáció

2.1 Fejlesztői környezet

A jogsifoglaló weboldal fejlesztéséhez több korszerű fejlesztőeszközt használtunk. Az adatbázis kialakításához a XAMPP fejlesztői környezetet alkalmaztuk, amelyen belül a MySQL adatbázis-kezelő rendszert használtuk az adatok tárolására és kezelésére. Az adatbázis adminisztrációját a phpMyAdmin felület segítette.

A backend fejlesztése Visual Studio 2022 környezetben történt, ahol .NET 8 technológiával készítettünk Web API alkalmazást C# programozási nyelven. Ez biztosította a kapcsolatot az adatbázis és a kliensoldali alkalmazás között.

A frontend fejlesztéséhez React könyvtárat használtunk JavaScript nyelven, a projekt futtatásához és kezeléséhez pedig a Vite build eszközt alkalmaztuk. A React komponensalapú felépítése lehetővé tette az átlátható és moduláris fejlesztést.

Ezeket a technológiákat azért választottuk, mert tanulmányaink során már megismertük őket, így hatékonyan tudtuk alkalmazni a projekt megvalósítása során.

2.2 Műszaki feltételek

Ahhoz, hogy a Jogsifoglaló alkalmazás hibátlanul működjön, a következőkre van szükség:

Szerver oldali működés*:*

Adatbázis: Kell egy MySQL szerver, hogy az adatokat (időpontok, felhasználók) legyen hol tárolni.

Futtató környezet: Mivel a backend .NET 8-ban készült szükségünk van a .NET 8 futtatókörnyezetre.

Internet: Az API-nak folyamatosan elérhetőnek kell lennie, hogy a frontend kommunikálni tudjon vele.

Felhasználói oldali működés:

Böngésző: Bármilyen modern böngésző megfelel (Chrome, Firefox).

Eszközök: Az oldal reszponzív, tehát nem csak asztali gépen, hanem tableten és okostelefonon is ugyanúgy használható.

Adatkapcsolat: A foglalások valós időben történnek, így stabil internetre van szükség a használata közben.

3. Csapatmunka

3.1 Feladatok elosztása

Feladat	**Felelős fejlesztő**
Adatbázis tervezése és implementálása	Csaba Bence
Backend (WebAPI) fejlesztése	Csaba Bence
Frontend (React/Vite) kialakítása	Takács Barnabás
Dokumentáció készítése	Csaba Bence & Takács Barnabás
Prezentáció és bemutató összeállítása	Takács Barnabás

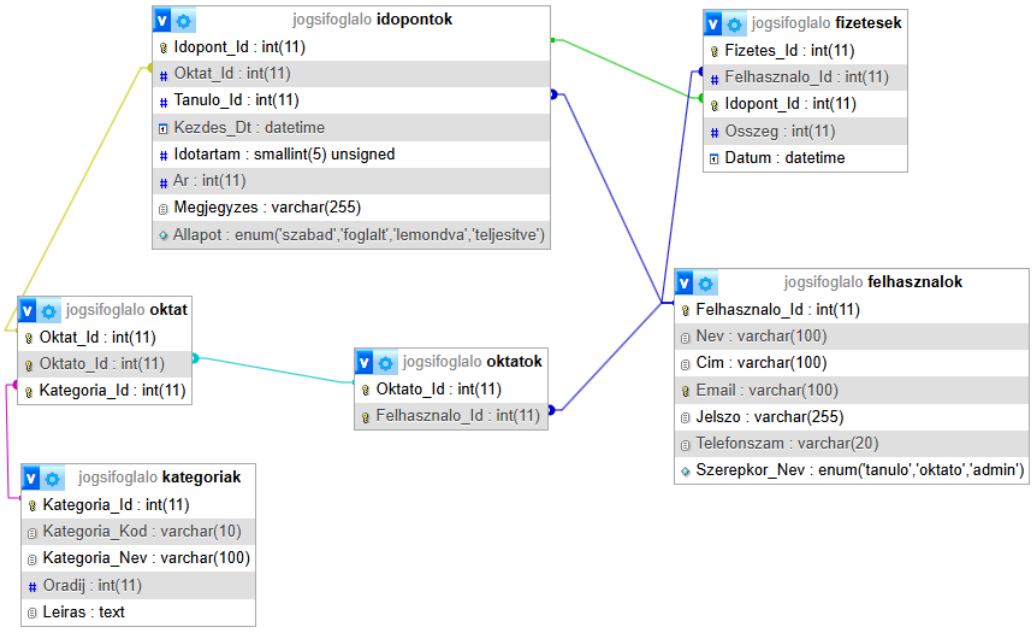
3.2 Kommunikáció

A projekt sikeres megvalósításához elengedhetetlen volt a folyamatos és hatékony kommunikáció. Mivel a fejlesztés több párhuzamos szálon futott, strukturált munkamenetet alakítottunk ki. A projekt fájljait és a dokumentációt is, végig Google Drive-on osztottuk meg egymással, a projekt és dokumentáció végleges verziója került fel Github-ra. Kommunikációra, napi színtű gyors információcsere és technikai elakadások azonnali megoldása érdekében messenger és discord felületeket használtuk. A legfontosabb tervezési döntéseket és a modulok integrációját az iskolai projektmunka órák keretében, személyesen egyeztettük.

4. Kialakított adatbázis és részletes bemutatása

4.1 Az adatbázis ER modell leírása

A rendszer az adatbázis-modell segítségével biztosítja az autósiskolai folyamatok hatékony adminisztrációját, az oktatók, a tanulók és a meghirdetett vezetési időpontok közötti kapcsolatok precíz nyilvántartását.



1. Ábra A Jogsifoglaló logikai adatbázisterve

Az adatbázis fizikai megvalósítását az alábbi diagram szemlélteti, amely tartalmazza a pontos adattípusokat és a táblák közötti kapcsolatok. Az absztrakt, fogalmi színtű összefüggéseket leíró ER-Modell a dokumentáció végén a mellékletek között található.

4.2 Egyedtípusok

Felhasználók (felhasznalok):

Ebben a táblában tároljuk a regisztrált felhasználókat és adataikat, legyen az tanuló, oktató vagy adminisztrátor.

Felhasznalo_Id: Int (Szám), Primary key (Elsődleges kulcs), Auto_increment (Automatikusan generált). Ez egy egyedi azonosító szám, ez alapján különbözteti meg a rendszer az embereket. Ezt a rendszer automatikusan generálja minden regisztrációnál.

Nev: Varchar(100) (Szöveg, ami max 100 karakterből állhat), Not Null (Nem lehet null a mező értéke). A felhasználó teljes neve, kötelező megadni.

Email: Varchar(100) (Szöveg, ami max 100 karakterből állhat), Unique (Egyedi), Not Null (Nem lehet null a mező értéke). Egyedi azonosításra szolgál, ennek segítségével jelentkezünk, majd be. Nem lehet két ugyanazzal az E-Mail címmel regisztrált fiók.

Jelszo: Varchar(255) (Szöveg, ami max 255 karakterből állhat), Not Null (Nem lehet null a mező értéke). BCrypt technológiával hash-elt, titkosított jelszó.

Szerepkor_Nev: Enum('tanulo','oktato','admin'). Meghatározott lista, ami korlátozza a választható rangokat. Az alap rang regisztráció után a 'tanulo'.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**Felhasznalo_Id**	int(11)			No	*None*		AUTO_INCREMENT
2	**Nev**	varchar(100)	utf8mb4_hungarian_ci		No	*None*		
3	**Cim**	varchar(100)	utf8mb4_hungarian_ci		Yes	*NULL*		
4	**Email**	varchar(100)	utf8mb4_hungarian_ci		No	*None*		
5	**Jelszo**	varchar(255)	utf8mb4_hungarian_ci		No	*None*		
6	**Telefonszam**	varchar(20)	utf8mb4_hungarian_ci		No	*None*		
7	**Szerepkor_Nev**	enum('tanulo', 'oktato', 'admin')	utf8mb4_hungarian_ci		No	tanulo		

2. Ábra A felhasználók tábla mezői és beállításai

Fizetések (fizetesek):

Ebben a táblában tároljuk a fizetéseket.

Fizetes_Id: Int (Szám), Primary key (Elsődleges kulcs), Auto_increment (Automatikusan generált). Ez egy egyedi azonosító szám, ez alapján különbözteti meg a rendszer a fizetéseket. Ezt a rendszer automatikusan generálja minden fizetésnél.

Felhasznalo_Id: Int (Szám), Foreign key (Idegen kulcs). Ezzel kapcsoljuk össze a fizetést a felhasználóval, mivel ez egy idegen kulcs, ami a Felhasználók táblára mutat. Amelyik felhasználó fizette ki az órát, annak a 'Felhasznalo_Id'-ja kerül be az adatbázisba.

Idopont_Id: Int (Szám), Foreign key (Idegen kulcs). Ezzel kapcsoljuk össze, melyik időpontot fizette ki az adott felhasználó.

Osszeg: Int (Szám), Not Null (Nem lehet null a mező értéke). Itt tároljuk az összeget, amit kifizette a tanuló.

Datum: Datetime (Ez egy időpont). A befizetés pillanatnyi ideje. Ezt nem mi adjuk meg, hanem az adatbázis magától adja meg az időpontot a 'current_timestamp()' beállítás miatt.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	**Fizetes_Id**	int(11)			No	*None*		AUTO_INCREMENT
☐	2	**Felhasznalo_Id**	int(11)			No	*None*		
☐	3	**Idopont_Id**	int(11)			No	*None*		
☐	4	**Osszeg**	int(11)			No	*None*		
☐	5	**Datum**	datetime			No	current_timestamp()		

3. Ábra A fizetés tábla mezői és beállításai

Időpontok (idopontok):

Ebben a táblában tároljuk az összes naptári bejegyzést legyen az szabad, foglalt vagy már levezetett óra. A fizetések tábla egy attribútumokkal rendelkező kapcsolótábla. Nemcsak összeköti a felhasználókat az időponttal, hanem saját adatokat is hordoz, amik csak az adott tranzakcióhoz tartoznak.

Idopont_Id: Int (Szám), Primary key (Elsődleges kulcs), Auto_increment (Automatikusan generált). Ez egy egyedi azonosító szám, ez alapján különbözteti meg a rendszer az időpontokat. Ezt a rendszer automatikusan generálja minden időpontnál.

Oktat_Id: Int (Szám), Foreign key (Idegen kulcs). Ezzel kapcsoljuk össze az időpontot a konkrét oktatási tevékenységekkel. Melyik oktató melyik kategóriát oktatja.

Tanulo_Id: Int (Szám), Foreign key (Idegen kulcs), Default Null (Alapból nincsen értéke a mezőnek). Itt tároljuk a foglaló tanuló azonosítóját, értelemszerűen alapból azért nulla, mivel amíg nincsen lefoglalva az adott óra, addig nem tudjuk ki fogja lefoglalni. A foglalás idején az óra típusa átvált szabadról foglaltra és bekerülő a tanuló azonosítója, aki lefoglalta az órát.

Kezdes_Dt: Datetime (Ez egy időpont), Not Null (Nem lehet null a mező értéke). A vezetési óra kezdő dátuma és pontos ideje.

Idotartam: Smallint (Rövid egész szám), Unsigned (Nem lehet negatív szám). Az óra hossza percekben megadva. Az alapértelmezett érték 120 perc, azaz 2 óra.

Ar: Int (Szám), Not Null (Nem lehet null a mező értéke). Az adott vezetési alkalom ára forintban.

Megjegyzés: Varchar(255) (Szöveg, ami max 255 karakterből állhat), Default Null (Alapból nincsen értéke a mezőnek). Ide kerülhetnek kiegészítő információk az óráról, mivel alapból nincsen értéke a mezőnek, nem is kötelező megadni semmilyen megjegyzést.

Allapot: Enum('szabad,'foglalt','lemondva','teljesítve'). Ez a mező határozza meg az időpont aktuális státuszát a naptárban. Alapból minden óra, ami létrejön 'szabad' állapotot kap.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**Idopont_Id**	int(11)			No	*None*		AUTO_INCREMENT
2	**Oktat_Id**	int(11)			No	*None*		
3	**Tanulo_Id**	int(11)			Yes	*NULL*		
4	**Kezdes_Dt**	datetime			No	*None*		
5	**Idotartam**	smallint(5)		UNSIGNED	No	120		
6	**Ar**	int(11)			No	*None*		
7	**Megjegyzes**	varchar(255)	utf8mb4_hungarian_ci		Yes	*NULL*		
8	**Allapot**	enum('szabad', 'foglalt', 'lemondva', 'teljesitve'...	utf8mb4_hungarian_ci		No	szabad		

4. Ábra Az időpontok tábla mezői és beállításai

Kategóriák (kategoriak):

Ebben a táblában tároljuk a választható jogosítvány típusokat és hozzájuk tartozó információkat.

Kategoria_Id: Int (Szám), Primary key (Elsődleges kulcs), Auto_increment (Automatikusan generált). Ez egy egyedi azonosító szám, ez alapján különbözteti meg a rendszer a kategóriákat. Ezt a rendszer automatikusan generálja minden kategóriánál.

Kategoria_Kod: Varchar(10) (Szöveg, ami max 10 karakterből állhat), Not Null (Nem lehet null a mező értéke). A jogosítvány kategória rövid jele.

Kategoria_Nev: Varchar(100) (Szöveg, ami max 100 karakterből állhat), Not Null (Nem lehet null a mező értéke). A kategória teljes megnevezése.

Óradíj: Int (Szám), Not Null (Nem lehet null a mező értéke). Az adott kategóriára vonatkozó alapvető díj forintban megadva (Ez csak egy összehasonlítási alap az oktatóknak).

Leiras: Text (Hosszabb szöveg), Not Null (Nem lehet null a mező értéke). Részletes tájékoztatás a kategóriáról.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**Kategoria_Id**	int(11)			No	*None*		AUTO_INCREMENT
2	**Kategoria_Kod**	varchar(10)	utf8mb4_hungarian_ci		No	*None*		
3	**Kategoria_Nev**	varchar(100)	utf8mb4_hungarian_ci		No	*None*		
4	**Oradij**	int(11)			No	*None*		
5	**Leiras**	text	utf8mb4_hungarian_ci		No	*None*		

5. Ábra A kategória tábla mezői és beállításai

Oktat (oktat):

Ez egy úgynevezett kapcsolótábla, amely az oktatók és az általuk tanítható kategóriák közötti kapcsolatot rögzíti.

Oktat_Id: Int (Szám), Primary key (Elsődleges kulcs), Auto_increment (Automatikusan generált). A kapcsolat egyedi azonosítója.

Oktato_Id: Int (Szám), Foreign key (Idegen kulcs). Hivatkozás az ’oktatok’ táblára, azaz megmondja, melyik oktatóról van szó.

Kategoria_Id: Int (Szám), Foreign key (Idegen kulcs). Hivatkozás a ’kategoriak’ táblára, azaz megmondja melyik kategóriát taníthatja az adott oktató.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	**Oktat_Id**	int(11)			No	*None*		AUTO_INCREMENT
☐	2	**Oktato_Id**	int(11)			No	*None*		
☐	3	**Kategoria_Id**	int(11)			No	*None*		

6. Ábra Az oktat kapcsolótábla mezői és beállításai

Oktatók (oktatok):

Ez a tábla választja szét a sima felhasználókat azoktól, akik oktatni is tudnak a rendszerben. Ezt nevezhetjük az oktatók „azonosító kártyájuknak”.

Oktato_Id: Int (Szám), Primary Key (Elsődleges kulcs), Auto_increment (Automatikusan generált). Az oktató saját egyedi azonosítója a rendszerben.

Felhasznalo_Id: Int (Szám), Foreign Key (Idegen kulcs). Ez köti össze az oktatót a ’felhasznalok’ táblával, így tudjuk meg a nevét és az elérhetőségeit.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
☐	1	**Oktato_Id**	int(11)			No	*None*		AUTO_INCREMENT
☐	2	**Felhasznalo_Id**	int(11)			No	*None*		

7. Ábra Az oktató kapcsolótábla mezői és beállításai

4.3 Az adatbázis kapcsolatok és kényszerfeltételek

Az adatbázis tervezésekor kiemelt figyelmet fordítottunk az adatok épségére. Ez azt jelenti, hogy a táblák nem csak egymás mellett vannak, hanem ’figyelnek’ egymásra, ezzel megakadályozzák, hogy hibás vagy felesleges adatok maradjanak a rendszerben.

Kapcsolatok:

Egy-az-egyhez (1:1) kapcsolat:

Ez a legritkább kapcsolat, akkor használjuk, ha egy adat szorosan és kizárólagosan csak egy másik adathoz tartozhat. Példa a rendszerben: A ’felhasznalok’ és az ’oktatok’ tábla kapcsolata. Egy felhasználónak csak egy oktatói profilja lehet és egy oktatói profil mögött is csak egyetlen regisztrált felhasználó állhat.

Egy-a-többhöz (1:N) kapcsolat:
Ez a leggyakoribb típus. Azt jelenti, hogy az egyik tábla egy sorához a másik tábla több sora is tartozhat. Példa a rendszerben: A ’felhasznalok’ (tanuló) és az ’idopontok’ kapcsolata.
Egy tanulóhoz több különböző vezetési időpont is tartozhat az idők folyamán, de egy konkrét időpontot egyszerre csak egy tanuló foglalhat le.

Több-a-többhöz (M:N) kapcsolat:
Ez akkor fordul elő, ha mindkét oldalról többszörös kapcsolódás lehetséges. Az adatbázis-kezelők ezt csak egy harmadik, úgynevezett kapcsolótábla segítségével tudják kezelni. Példa a rendszerben: Az ’oktatok’ és a ’kategoriak’ kapcsolata. Egy oktató több kategóriát is taníthat és egy kategóriát is több különböző oktató taníthat.

Forrás tábla	**Cél tábla**	**Kapcsolat típusa**	**Kapcsoló mező (Foreign Key)**	**Magyarázat**
felhasznalok	oktatok	**1:1**	Felhasznalo_Id	Egy felhasználónak egy oktatói profilja lehet.
felhasznalok	idopontok	**1:N**	Tanulo_Id	Egy tanuló több órát is lefoglalhat.
oktatok	oktat	**1:N**	Oktato_Id	Egy oktató több kategóriát is taníthat.
kategoriak	oktat	**1:N**	Kategoria_Id	Egy kategóriához több oktató is tartozhat.
idopontok	fizetesek	**1:N**	Idopont_Id	Egy vezetési órához tartozhat befizetés.
felhasznalok	fizetesek	**1:N**	Felhasznalo_Id	Egy felhasználó több befizetést is indíthat.

Kényszerített törlés (ON DELETE CASCADE):

A vizsgaremek fejlesztése során kiemelt szempont volt az adatintegritás. Nem engedhetjük meg, hogy ha törlünk egy felhasználót, akkor a hozzá órarendi bejegyzések bent maradjanak az adatbázisban, mert ez hibás működéshez vezet. Erre szolgál az ON DELETE CASCADE (vízeséssszerű törlés) beállítása az idegen kulcsokon. Így, ha törlünk egy tanulót vagy oktatót minden plusz adat, ami hozzátartozott például az órák, azok mind törlődnek a felhasználóval együtt. Erre lehetett volna egy másik megoldás is, hogy a felhasználó tábla végére, létrehoztunk volna egy bool elemet, aminek igaz vagy hamis az értéke és így nem töröljük a felhasználót, hanem deaktiváljuk és ezzel megmaradnak az órarendi bejegyzések és minden hozzákapcsolt adat.

ON DELETE CASCADE működési elve:

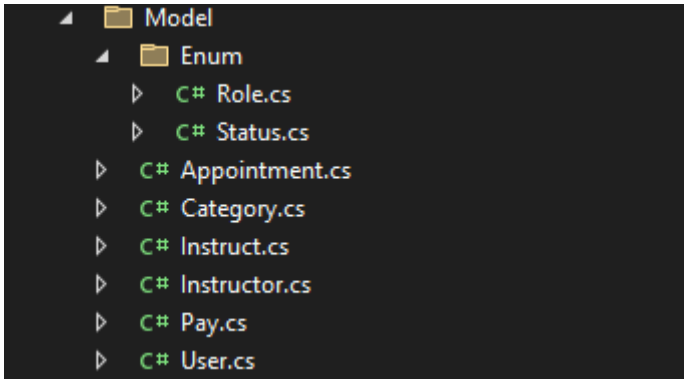
Amikor a főtáblából törlünk egy adatot, legyen ez a ’felhasznalo’ tábla, a törlés során a program végig megy a kapcsolatokon és kitöröl mindent, ami a felhasználóhoz van kapcsolva. Amikor törlünk egy tanulót a rendszer azonnal törli a hozzátartozó összes bejegyzést a fizetés táblából és az összes foglalt órát is. Ennek az előnye, nem maradnak felesleges sorok, amik foglalnák a tárhelyet. Emellett megakadályozza, hogy olyan adatokra hivatkozzunk, amik már nem léteznek

5. Backend felépítése és projektstruktúra (C# / .NET Core Web API)

Az alkalmazás a Client-Server architektúrára épül, ahol a frontend és a backend élesen elkülönül egymástól. A rendszer „agya” a backend, amely az üzleti logikáért és az adatbázissal való kommunikációért felel. A szerver Restful végpontokat biztosít, amelyen keresztül a frontend JSON kéri le vagy küldi el az adatokat. Entity Framework Core-t használunk az adatbázis és a C# kód összekapcsolására. Ez azért jó, mert a táblák C# osztályonként jelennek meg, így a lekérdezések biztonságosabbak és gyorsabbak. A backendben történik a jelszavak titkosítása a már említett BCrypt hash-eléssel, emellett itt történik a jogosultságok kezelése és ellenőrzése. Bár a projekt alapja az ASP.NET Core MVC keretrendszer, a "View" réteget leválasztottuk, és egy modern Web API architektúrát alkalmaztunk. A kód tisztasága és tesztelhetősége érdekében bevezettük a Repository Pattern-t.

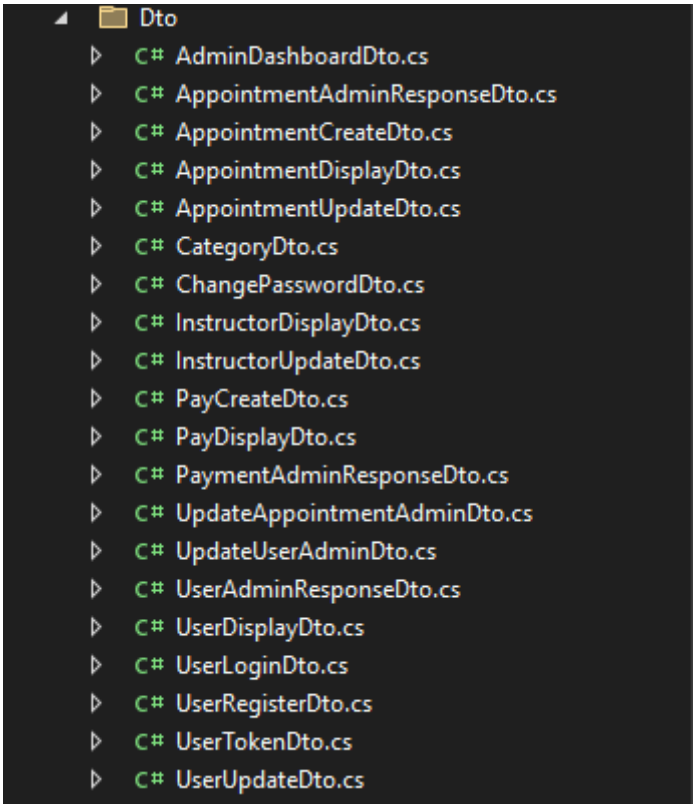
5.1 A rétegek feladatai

Models (Entitások): Ezek az osztályok tükrözik az adatbázis tábláit, az Entity Framework ezek alapján hozza létre a lekérdezéseket. Ezenkívül a modell mappában található meg egy Enum mappa, amiben az adatbázisban látott Enum értékeit raktuk a jobb átláthatóság érdekében.



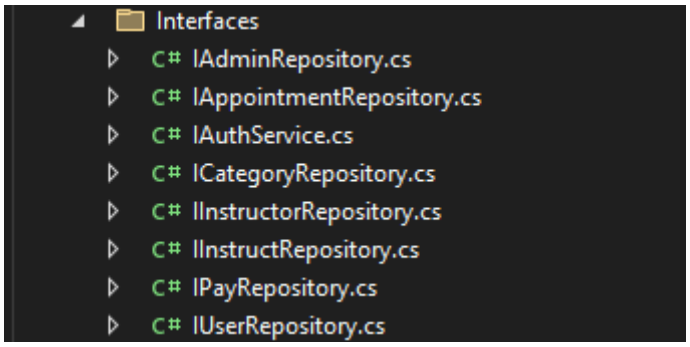
8. Ábra A models mappa tartalma

DTO (Data Transfer Objects): A DTO-kat azért használjuk, hogy elrejtsük az érzékeny adatokat. Például a 'UserDisplayDto' tartalmazza a felhasználó összes adatát, de nem tartalmazza a jelszót, amit a szerver kap meg. Ez biztonsági szempontból alapvető.



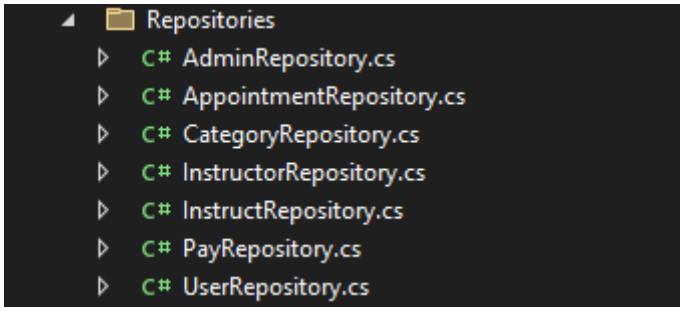
9. Ábra A dto mappa tartalma

Interfaces (Interfészek): Az interfészek határozzák meg milyen műveletek végezhetünk, de azt nem mondják meg, hogyan.



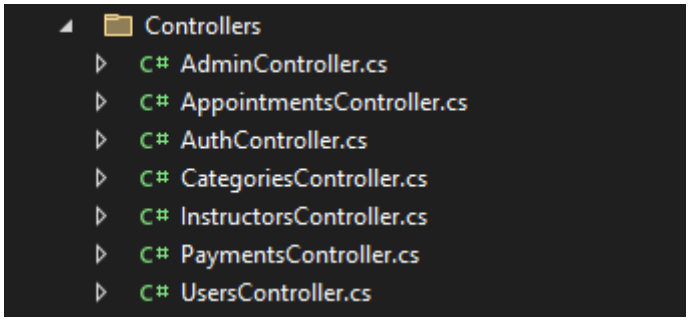
10. Ábra Az interfaces mappa tartalma

Repositories (tárolók): Ez a réteg tartalmazza a konkrét SQL lekérdezéseket. Itt választjuk el az üzleti logikát az adatbázis-kezeléstől. Ha később esetleg másik adatbázisra váltanánk, csak a Repository-t kellene átírni, a Controllert nem.



11. Ábra A repositories mappa tartalma

Controllers (Kontrollerek): A Controller igazából csak egy forgalomirányító. Megkapja a kérést a frontendetől, meghívja a megfelelő Repository-t és visszaadja az eredményt általában DTO formátumban.

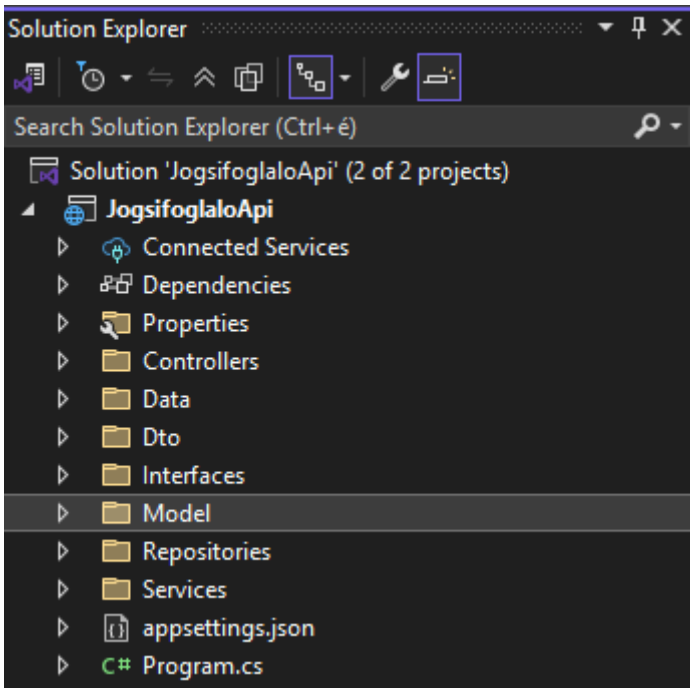


12. Ábra A controllers mappa tartalma

Ezt a felépítést többrétegű architektúrának hívják. Az alkalmazás fejlesztése során a szétválasztott felelősségek elvét követtük. Ez azt jelenti, hogy a kód különböző logikai rétegekre van osztva, így a rendszer skálázható és könnyen karbantartható. Fontos megjegyezni, hogy bár egy autósiskolai rendszer megvalósítható lett volna egy egyszerűbb, kevésbé tagolt struktúrával is, mi tudatosan döntöttünk a többrétegű architektúra mellett. Szerettük volna kipróbálni, hogyan működik a valóságban megfelelő, skálázható rendszer. Ennek előnye, hogy sokkal átláthatóbb a kód. Fent látható mappákon kívül van még egy Data mappánk, ebben található a JogsifoglaloContext osztály, itt történik az alacsony színtű adatbázis-kezelés. Van még egy Services mappa, amiben található az AuthService osztály, ahol regisztrációnál a jelszó titkosítása történik, bejelentkezésnél a jelszó visszafejtése, hogy betudjunk jelentkezni, emellett itt jön létre a token, amit a böngészőben elmentünk a frontend segítségével, ezzel a tokennel tudjuk igazolni magunkat. Ezeken kívül található a program osztály, itt történik a Dependecy injeciton regisztrációja, ahol megmondjuk a rendszernek, hogy melyik interfészhez melyik osztály tartozik. Indításkor ez a fájl állítja össze az alkalmazást, a beállításokat pedig az

appsettings.development.json fájlból olvassa ki. Az említett appsettings fájlban tároljuk a Connection Stringet, amivel megmondjuk a .NET-nek hova csatlakozzon. A Connection Stringet a fejlesztői környezetben használtam, de éles üzemben, ezeket az adatokat biztonságosabb helyen kellene tárolni.

Itt található meg a JWT (JSON Web Token) beállításai, itt tároljuk a titkos kulcsot, amivel a tokeneket aláírjuk. Ezt a kulcsot szigorúan titokban kell tartani. A program osztályban állítottuk be a CORS politikát is. Röviden ez azért kellett nekünk, mert a böngésző biztonsági okokból tiltja, hogy a Frontend kéréseket küldjön egy másik portón futó Backendnek. A CORS szabályok megfelelő konfigurálásával engedélyeztük a biztonságos kommunikációt a két réteg között



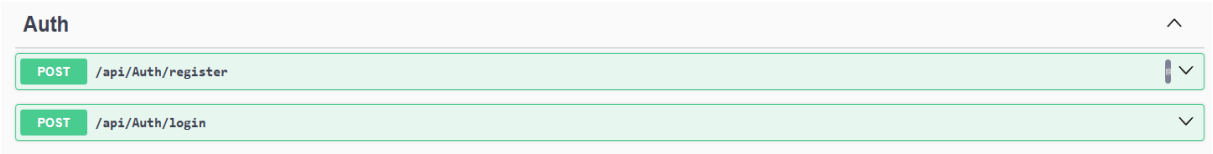
13. Ábra Az API mappa szerkezete

5.2 A kontrollerek részletes bemutatása

Minden kontrollerünk működését a Swagger felületén keresztül dokumentáltuk és teszteltük.

Hitelesítés (AuthController): Ez a kontroller kezeli a felhasználók belépési pontjait. Nem közvetlenül az adatbázishoz beszél, hanem az AuthService-en keresztül végzi a logikát.

Főbb metódusok: Post: /api/Auth/login (Bejelentkezés és Token generálás), Post: /api/Auth/register (Új felhasználó létrehozása BCrypt hasheléssel).



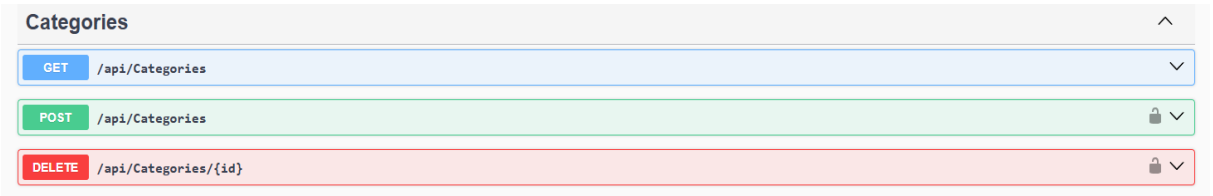
14. Ábra Az AuthController végpontjai a Swagger felületén

Időpontkezelés (AppointmentsController): A rendszer legfontosabb része itt történik, a foglalás kezelése. Főbb metódusok: Get: /api/Appointments/available (Lekéri az összes szabad órát), Put: /api/Appointments/book/{id} (Lefoglalunk egy adott időpontot), Post: /api/Appointments/create (Létrehozza az oktató a szabad órát).

15. Ábra Az Appointments végpontjai a Swagger felületén

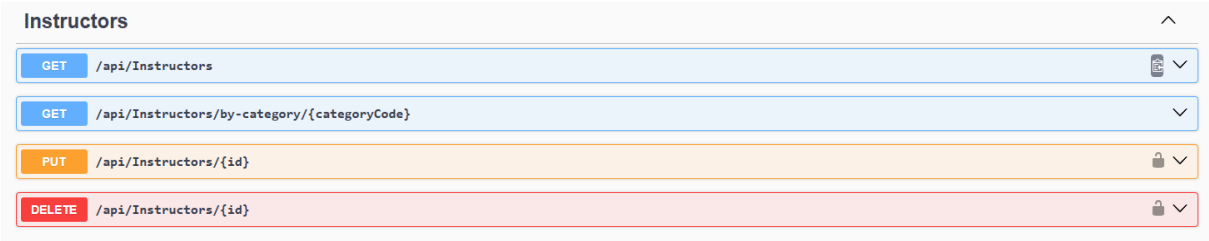
Kategóriák (CategoriesController): Itt tálalhatóak a kategóriák, amiket oktatnak az oktatók.

Főbb metódusok: Get: /api/Categories (Visszakapjuk az összes oktatási kategóriát és a kategóriák adatait).



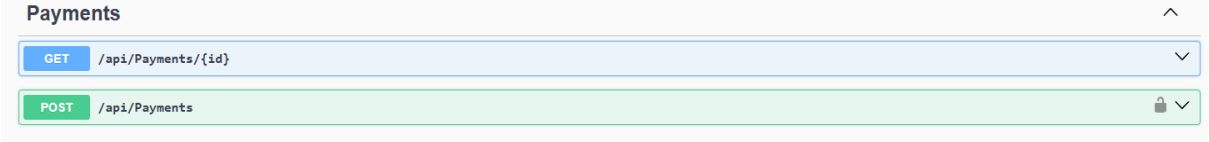
16. Ábra A Categories kontroller a Swagger felületén

Oktatók (InstructorsController): Itt találhatóak az oktatók és adataik. Főbb metódusok: Get: /api/Instructors/by-category/{categoryCode} (Ezzel a metódussal kitudjak keresni az oktatókat az alapján, hogy milyen kategóriában oktatnak, ezt használjuk a naptárnál amikor keressünk), Put: /api/Instructors/{id} (Itt tudjuk módosítani az oktató adatait, emellett itt tudunk új oktatási kategóriát hozzáadni az oktatóhoz, ha a későbbiekben más kategóriát akarna oktatni).



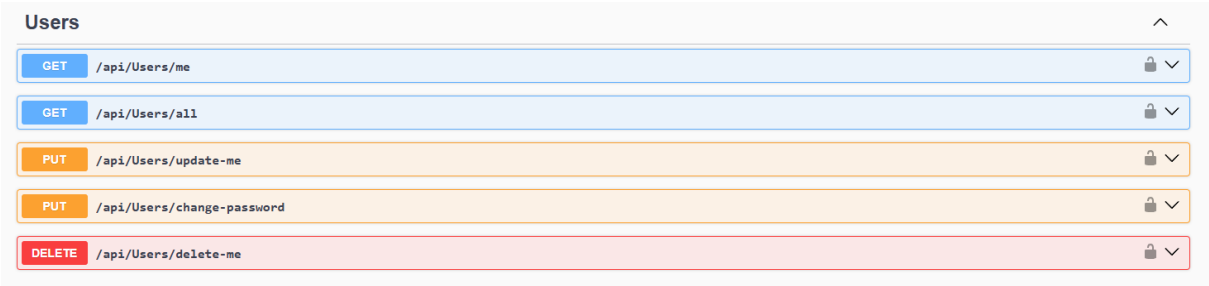
17. Ábra A Instuctors végpontjai a Swagger felületén

Fizetés (PaymentsController): Ebben a kontrollerben a fizetés történik. Főbb metódusok: Post: /api/Payments (Ezzel a metódussal tudunk létrehozni új fizetést).



18. Ábra A Payments végpontjai a Swagger felületén

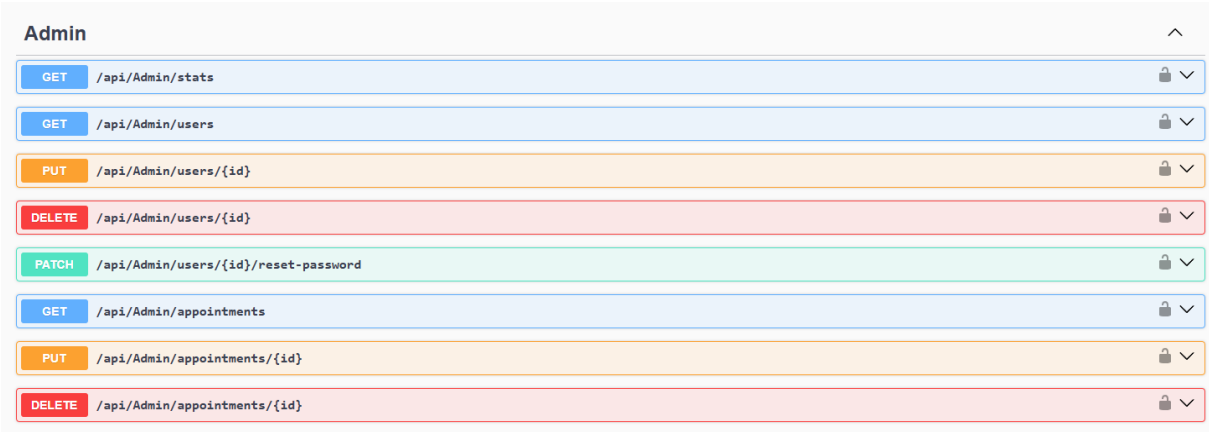
Felhasználók (UsersController): Szinten egy nagyon fontos kontroller, itt történik a felhasználok lekérése, a saját adatok módosítása és a saját profil törlése. Főbb metódusok: Get: /api/Users/me (Visszakapjuk a saját adatainkat), Put: /api/Users/update-me (Itt a saját adatainkat tudjuk módosítani), Delete: /api/Users/delete-me (Saját profil törlése)



19. Ábra A Users végpontjai a Swagger felületén

Adminisztráció (AdminController): Ez a kontroller felelős a rendszer globális felügyeletéért, biztosítja, hogy az iskola vezetője vagy a rendszergazda teljes rálátást kapjon a folyamatokra, és be tudjon avatkozni, ha szükséges

Főbb metódusok: Get: /api/Admin/stats (Statisztikákat ad vissza), Put: /api/Admin/users{id} (Felhasználok adatainak módosítása), Delete: /api/Admin/users/{id} (Törölni tudunk felhasználókat), Patch: /api/Admin/users/{id}/reset-password (Megtudjuk változtatni adott felhasználó jelszavát, ha elfelejtené), Put: /api/Admin/appointments{id} (Időpontok adatainak módosítása), Delete: /api/Admin/appointment/{id} (Törölni tudunk időpontokat),



20. Ábra Az Admin végpontjai a Swagger felületén

5.3 Biztonság és Jogosultságkezelés (Authorization & Authentication)

Egy autósiskolai rendszernél kritikus, hogy egy tanuló ne tudja átírni a saját befizetéseit, vagy ne lásson bele mások adataiba. Erre szolgál a hitelesítési rendszerünk. A folyamat két lépcsőből áll: az Authentication (hitelesítés) során a rendszer a bejelentkezéskor meggyőződik a felhasználó kilétéről és kiállítja a JWT tokent, majd az Authorization (engedélyezés) során szerepkör alapján adunk hozzáférést a végpontokhoz. Ehhez a kontroller metódusai felett az [Authorize(Roles = "")] attribútumot használtuk. Ha érkezik egy kérés, a backend először kicsomagolja a JWT tokent, megnézi a benne lévő role (szerepkör) claim-et és ha nem egyezik az elvárttal, azonnal elutasítja a kérést és visszaad egy 403 hibakódot.

```
[Authorize(Roles = "admin,oktato")]
```

21. Ábra Authorize címke

6. Frontend megvalósítása (React & Vite)

Az alkalmazás felhasználói felülete modern, komponens alapú megközelítéssel készült. A választásunk a React keretrendszerre esett, a fejlesztői környezetet pedig a Vite biztosította.

6.1 React és a Vite

A Reactot a nagy közösségi támogatás és a virtuális DOM (Document Object Model) technológia miatt választottuk, ami villámgyors felületi frissítéseket tesz lehetővé anélkül, hogy az egész oldalt újra kellene tölteni. A Vite használata mellett a gyors fejlesztési idő és a modern modulkezelés miatt döntöttünk.

6.2 Alkalmazott külső könyvtárak és függőségek

A fejlesztés során több népszerű és megbízható könyvtárat integráltunk a projektbe, hogy növeljük a hatékonyságot és javítsuk a felhasználói élményt.

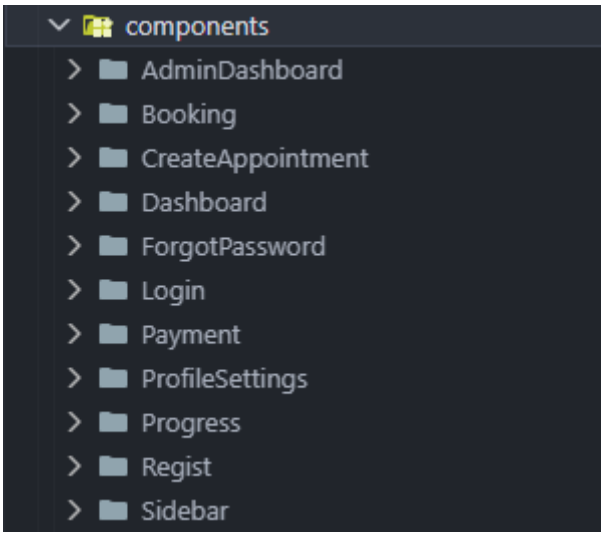
Axios: Ez a könyvtár felel a Frontend és a Backend közötti zökkenőmentes kommunikációért. Az Axios segítségével aszinkron HTTP kéréseket (GET, POST, PUT, DELETE) küldünk az API végpontjainkra. Előnye a beépített JSON adatkezelés és a kérések/válaszok egyszerűbb elfogása, ami lehetővé teszi a hitelesítési tokenek automatikus csatolását.

React-Router-Dom: A modern Single Page Application (SPA) működés elengedhetetlen kelléke. Ennek segítségével valósítottuk meg az oldalak közötti navigációt anélkül, hogy a böngészőnek újra kellene töltenie a teljes dokumentumot. Ez biztosítja a folyékony felhasználói élményt és a tiszta URL-struktúrát a különböző funkciók elérésekor.

SweetAlert2: A felhasználói visszajelzések vizuális megjelenítésére ezt a könyvtárat választottuk a gyári böngészőablakok helyett. Segítségével esztétikus, reszponzív felugró ablakokat készítettünk, amelyek azonnali visszajelzést adnak a műveletek sikerességéről vagy esetleges hibákról.

6.3 Komponens alapú felépítés

Az alkalmazás felhasználói felülete komponens-alapú architektúrára épül, amely a React.js technológiát alkalmazza. A rendszer egyoldalas webalkalmazásként (Single Page Application) működik, ezáltal az oldalak közötti navigáció az oldal teljes újra töltésé nélkül, dinamikusan megy végbe, minimalizálva a hálózati terhelést és növelve a felhasználói élményt. A frontend logikai és vizuális felépítése az alábbi főbb modulokra bontható:



22. Ábra Komponensek mappa tartalma

6.4 Állapotkezelés és Kliensoldali Útválasztás

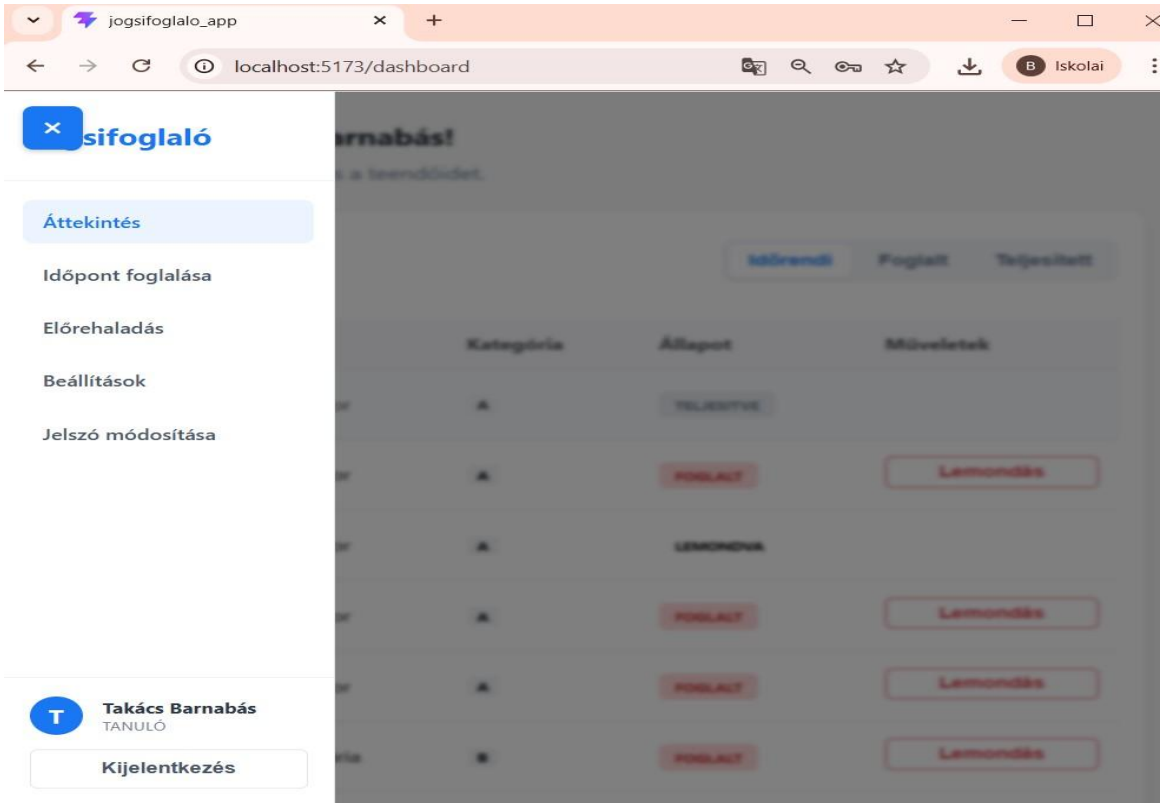
Az alkalmazás gyökérkomponense az App.jsx, amely a react-router-dom könyvtár segítségével definiálja a kliensoldali végpontokat. A rendszer biztonsági magját egy egyedi ProtectedRoute komponens, valamint a globális állapotot biztosító AuthContext alkotja. A Context API felel a localStorage-ben tárolt JWT (JSON Web Token) kezeléséért. A tokenből dekódolt adatok (például a felhasználó azonosítója és szerepköre) alapján a rendszer valós időben engedélyezi vagy tiltja le a védett útvonalakhoz való hozzáférést, szükség esetén átirányítva az illetéktelen kéréseket.

6.5 Autentikációs és Validációs Modulok

Az azonosítási folyamatok aszinkron Axios hívásokkal kommunikálnak a backend szerverrel. A regisztrációs modul (Regisztrációs oldal) kiemelkedő technikai jellemzője a kiterjedt, frontend oldali validáció. A komponens reguláris kifejezések (Regex) használatával a beküldés előtt ellenőrzi az e-mail cím formátumát, a magyar névszabályokat (nagy kezdőbetűk), a telefonszám helyességét, a jelszó komplexitását (hossz, kis- és nagybetűk, számok, speciális karakterek), továbbá beépített JavaScript dátumkezeléssel garantálja a 14 éves korhatár betartását. A hibakezelést és a felhasználói visszajelzéseket a SweetAlert2 modális ablakai végzik.

6.6 Reszponzív Navigációs Rendszer

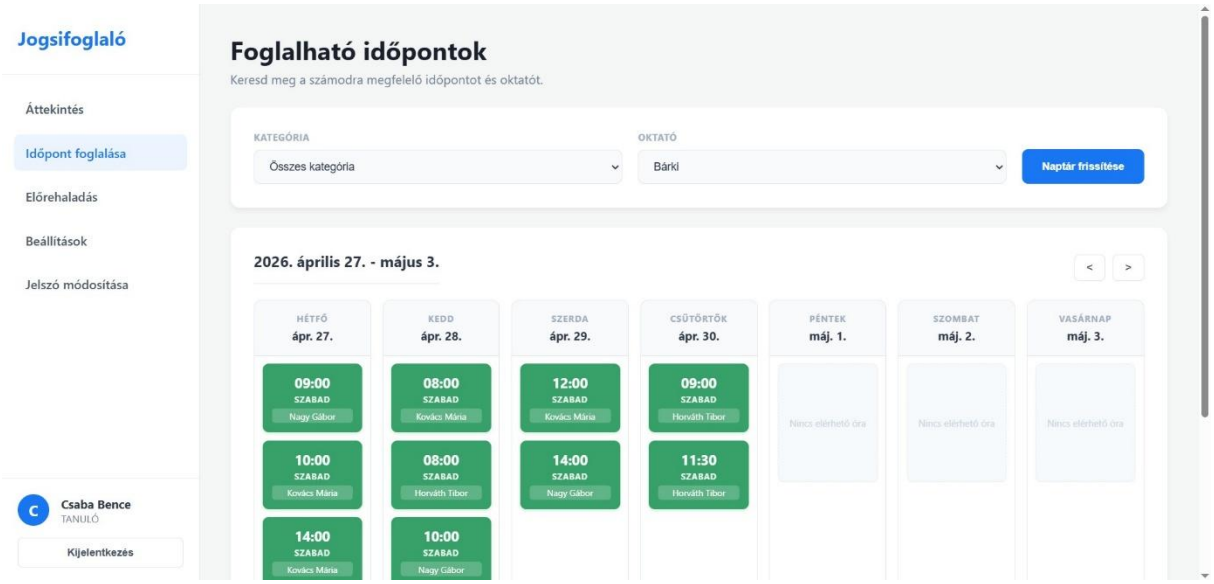
A menürendszer egy dinamikusan renderelt oldalsáv. A useLocation hook segítségével figyeli az aktuális URL-t, aktív állapotra váltva a megfelelő menüpontot. A komponens jogosultság-alapú renderelést használ: az aktuális felhasználó profiljától függően (tanuló, oktató vagy adminisztrátor) csak a releváns navigációs elemeket jeleníti meg. Kisebb képernyőfelbontás esetén a Sidebar automatikusan összecsukódik, és egy „hamburger” gombbal, valamint sötétítő háttérrel (overlay) támogatott, becsúszó mobilmenüként funkcionál.



23. Ábra Navigációs Rendszer

6.7 Naptár és Foglalási Logika

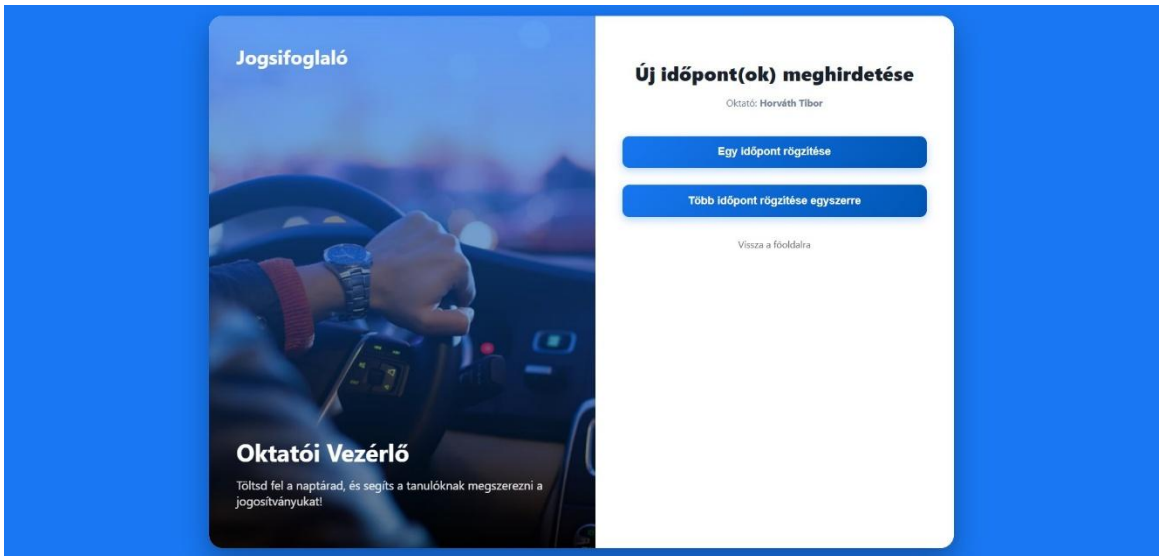
A tanulók számára dedikált foglalási felület egy egyedi logikájú, dinamikus 7 napos naptárrácsot generál. A rendszer a getMonday algoritmussal határozza meg az aktuális hét kezdőnapját, ehhez viszonyítva építi fel az időpontokat. A komponens összetett kliensoldali szűrést alkalmaz: a backendről kapott nyers adatokat azonnal leszűri a kiválasztott kategóriára és oktatóra, de ami a legfontosabb, a new Date() összehasonlítások segítségével a múltbéli időpontokat automatikusan eltávolítja a megjelenítendő elemek közül. Emellett a hétváltó (lapozó) gombok logikája letiltja a múlt heti naptárnézetek betöltését.



24. Ábra Naptár

6.8 Időpont-generátor Modul

Az oktatói felület kétféle rögzítési metódust (szimpla és tömbösített) tesz lehetővé a React állapotok (state) módosításával. Több időpont rögzítése esetén a rendszer egy iterálható memóriatömböt (multiData) használ, amelyből dinamikusan adhatók hozzá vagy törölhetők az űrlapsorok. A modul kritikus eleme a biztonságos adatrögzítést garantáló dupla ütközésvizsgálat: a POST kérések elküldése előtt a kód a frontenden ellenőrzi az űrlap elemeinek belső átfedését, majd a már adatbázisban szereplő foglalásokkal is összeveti azokat (időtartam-kalkuláció alapján), megelőzve a dupla rögzítést. A mentés véglegesítése optimalizált módon, Promise.all() metódussal fut le.

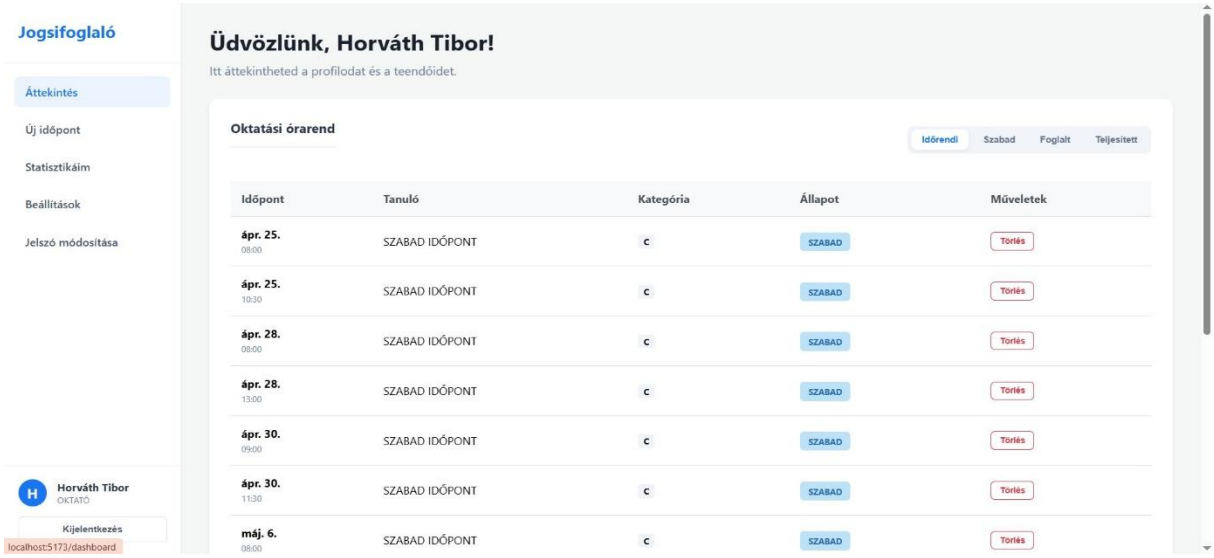


25. Ábra Időpont meghirdetése

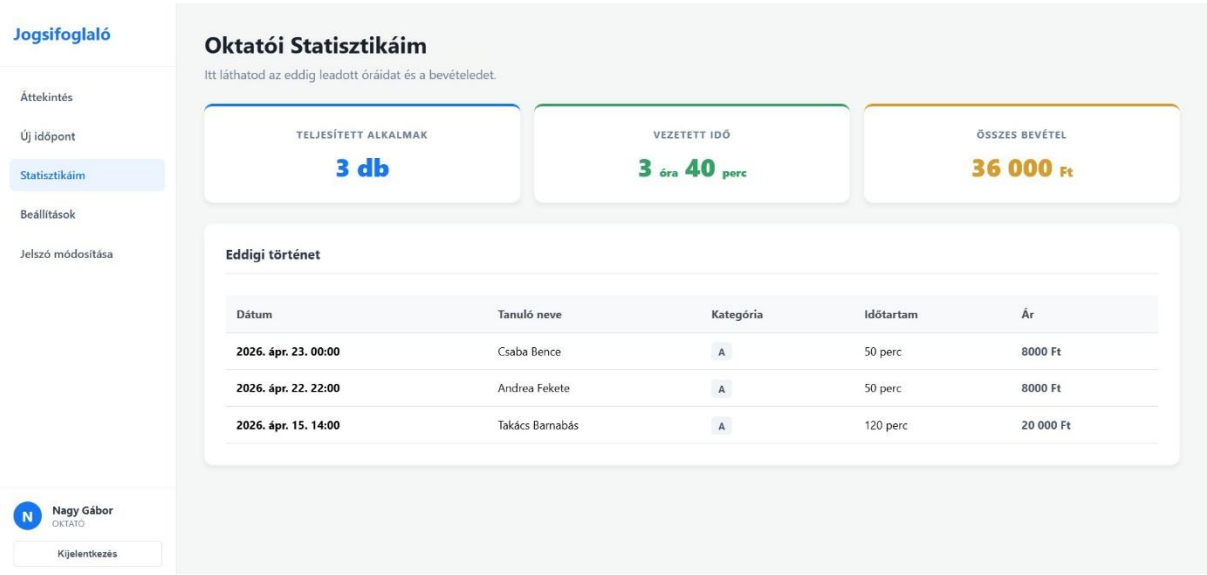
6.9 Áttekintő és Vizualizációs Panelek

Dashboard: Interaktív vezérlőpult, amely a felhasználó típusától függően hívja meg a megfelelő API végpontot. Az adatokat egy saját metódus (getFilteredLessons) segítségével azonnal, hálózati késleltetés nélkül rendezi és szűri (időrendi, szabad, foglalt, teljesített). Az oktatók erről a felületről kezdeményezhetik a lezajlott órák igazolását.

Progress: A statisztikai vizualizációért felelős modul. A nyers adathalmazt funkcionális JavaScript tömbműveletekkel (reduce, filter) dolgozza fel. Kiszámítja az elköltött/keresett összegeket, és a levezetett percek összegét átalakítja óra-perc formátummá, mindezt dinamikusan, oktatott kategóriák (pl. A, B, C) szerinti bontásban.



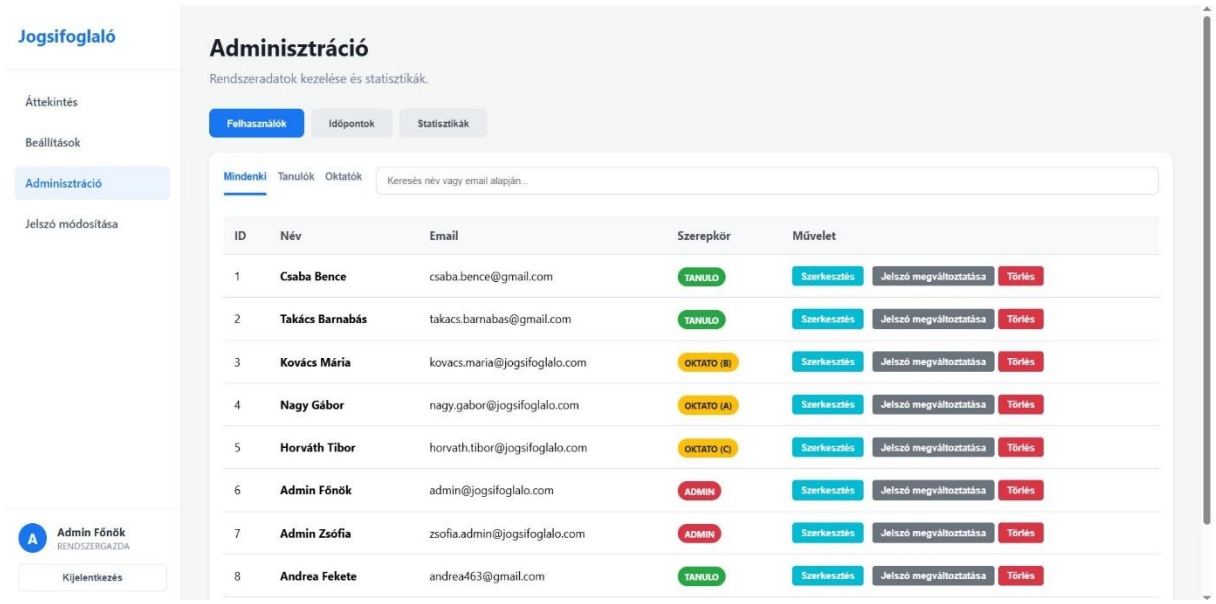
26. Ábra Vezérlőpult



27. Ábra Statisztikák

6.10 Adminisztrációs Rendszer

A legkomplexebb adatkezelési felület, amely belső, fül-alapú (tab) navigációval operál. Párhuzamosan több végpontról (felhasználók, instruktorok, időpontok, aggregált statisztikák) kéri le az adatokat, majd a kliensoldalon relációsan összekapcsolja azokat (például az oktatói azonosítókat és kategóriákat a felhasználói fiókokkal). A modul dedikált kereső- és szűrőmezőkkel, valamint beépített felugró (modal) szerkesztőablakkal rendelkezik. A módosítások során (pl. jelszó-visszaállítás vagy felhasználó törlése) fellépő szerveroldali megszakításokat (például adatbázis-kényszerek miatti 400-as vagy 500-as hibákat) a rendszer a try-catch blokkokon belül lekapja, és a SweetAlert2 segítségével felhasználóbarát, informatív módon jeleníti meg a felületen.



28. Ábra Adminisztrációs felület

6.11 Stíluslapok és Reszponzív Kialakítás (CSS)

Az alkalmazás vizuális interfésze moduláris CSS architektúrára épül, amely komponens-szintű elkülönítéssel akadályozza meg a stílusok globális ütközését. Az elrendezés alappilléreit a CSS Flexbox és CSS Grid technológiák biztosítják. Az összes felület mobilra optimalizált: a @media lekérdezések (media queries) segítségével a többoszlopos táblázatok, az osztott képernyős (split-container) űrlapok és a naptár rácsok is automatikusan újrarendeződnek, így az alkalmazás táblagépen és okostelefonon is kompromisszummentes felhasználói élményt nyújt.

7. Tesztelés

A szoftverfejlesztési folyamat elengedhetetlen része a minőségbiztosítás. Az alkalmazás tesztelését két szinten végeztük el: automatizált backend egységtesztekkel (Unit Tests) és manuális frontend teszteléssel.

7.1 Backend egységtesztek (Unit Tests)

A backend üzleti logikájának megbízhatóságát automatizált egységtesztekkel (Unit Tests) támogattuk meg. A teszteléshez az xUnit keretrendszert választottuk, amely a .NET ökoszisztéma egyik legnépszerűbb tesztelő eszköze. A tesztek írásakor az AAA mintát követtük, ami segít a tesztesetek tiszta tagolásában.

Arrange (Előkészítés): Itt állítottuk be a tesztkörnyezetet és a szükséges adatokat.

Act (Végrehajtás): Itt hívtuk meg a tesztelni kívánt kontroller metódusát.

Assert (Ellenőrzés): Itt vizsgáltuk meg, hogy a kapott válasz (Státuszkód) megegyezik-e az elvárttal.

Mivel az egységtesztek lényege az izoláció, a Moq könyvtár segítségével „szimuláltuk" (mockoltuk) a Repository réteget. Így a tesztek futtatásához nem volt szükség élő adatbázis-kapcsolatra, és kizárólag a kontroller logikáját tudtuk ellenőrizni. Teszteltük például, hogy a GetMe metódus megfelelően visszaadja-e a bejelentkezett felhasználó adatait (200 OK), vagy hiba esetén a megfelelő státuszkódot küldi-e.

```
[TestMethod]
0 references
public async Task GetProfile_ValidUser_ReturnsOk()
{
    var user = new User { Felhasznalo_Id = 1, Nev = "Teszt Elek", Email = "teszt@gmail.com", Jelszo = "hash",
    Telefonszam = "0630", Szerepkor_Nev = Role.tanulo };
    _mockRepo.Setup(r => r.GetByIdAsync(1)).ReturnsAsync(user);

    var result = await _controller.GetMyProfile();

    var okResult = result.Result as OkObjectResult;
    Assert.IsNotNull(okResult);
    var displayDto = okResult.Value as UserDisplayDto;
    Assert.AreEqual("Teszt Elek", displayDto?.Nev);
}
```

29. Ábra GetMe metódus sikeres tesztje

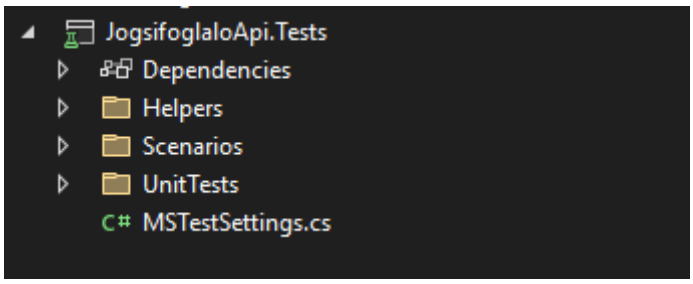
7.2 Forgatókönyv alapú tesztelés (Scenario)

A forgatókönyv alapú tesztelés során a rendszer teljes működési folyamatait vizsgáltuk a felhasználói élmény és az üzleti logika szempontjából. A tesztelés célja annak igazolása volt, hogy a Frontend és a Backend közötti adatcsere zökkenőmentes, és a rendszer képes kezelni a valós használat során felmerülő élethelyzeteket.

Tanuló folyamat tesztelése: Ebben a tesztben egy tanuló útját a regisztrációtól az időpontfoglalásig és fizetésig modellezük le. A teszt ellenőrzi a sikeres regisztrációt, majd a hitelesített kontextus beállítása után megkísérel egy szabad időpontot lefoglalni és kifizetni.
Itt teszteljük az AuthController és a PaymentsController közötti együttműködést. A teszt garantálja, hogy egy tanuló csak érvényes azonosítóval (Token) tud kifizetni egy létező, szabad időpontot.

Oktatói folyamat tesztelése: Az oktatói modul tesztelése a profilkezelésre és a jogosultságok védelmére fókuszál. Ellenőrizzük, hogy az oktató képes-e frissíteni a saját adatait. A forgatókönyv tartalmaz egy kritikus biztonsági tesztet is: megvizsgálja, hogy ha az „A" oktató megpróbálja módosítani a „B" oktató profilját, a rendszer helyesen 403 Forbidden vagy BadRequest választ ad-e. Ez garantálja az adatok sértetlenségét.

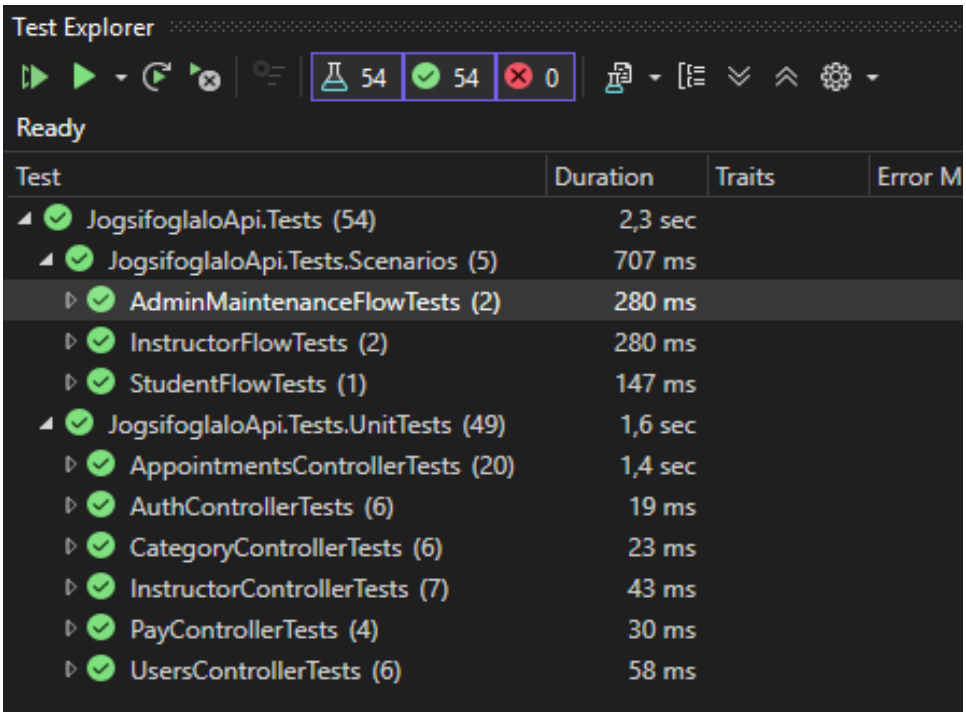
Adminisztrátori karbantartási folyamat: Az adminisztrátori tesztek a rendszer felügyeleti funkcióit vizsgálják. A forgatókönyv szimulálja, ahogy az Adminisztrátor lekéri a rendszer statisztikáit, majd elvégez egy moderációs műveletet (például felhasználó törlését). Teszteltük azt az esetet is, amikor az adminisztrátor egy nem létező felhasználót próbál törölni – a rendszernek ilyenkor is stabilnak kell maradnia és NotFound hibát kell adnia.



30. Ábra A Test mappa szerkezete

7.3 Tesztek futtatása és eredmények

A teszteket a Visual Studio Test Explorer felületén futtattuk. A sikeres lefutást a metódusok melletti zöld jelzések igazolják. Az automatizált tesztek garantálják, hogy egy későbbi fejlesztés vagy módosítás során nem rontjuk el a már meglévő, jól működő funkciókat.



31. Ábra A sikeresen lefutott backend tesztek listája

8. Telepítési útmutató

A rendszer három fő részegységből áll: az adatbázis-szerverből, a C# .NET alapú háttérszolgáltatásból (Backend) és a React alapú felhasználói felületből (Frontend). A szoftver futtatásához az alábbi lépések elvégzése szükséges.

8.1 Adatbázis konfigurálása (XAMPP)

Az alkalmazás az adatok tárolásához MySQL adatbázist használ, melyet a legegyszerűbben XAMPP környezetben lehet futtatni.

Indítsa el a XAMPP Control Panel alkalmazást.

Indítsa el az Apache és a MySQL modulokat a „Start” gombok segítségével.

Nyisson meg egy böngészőt, és lépjen a localhost/phpmyadmin felületre.

Az „Importálás” fülön tallózza be a projekthez mellékelt .sql fájlt, majd kattintson az „Indítás” gombra az adatbázis, a táblák és alapértelmezett adatok létrehozásához.

8.2 Backend (C# .NET API) indítása

A backend felel az üzleti logikáért és az adatbázissal való kapcsolattartásért.

Adatbázis-kapcsolat ellenőrzése: Nyissa meg a backend mappájában található appsettings.json fájlt. Ellenőrizze a ConnectionStrings részt.

Az alkalmazás alapértelmezetten a 3306-os porton keresi a MySQL szervert. Amennyiben a XAMPP-ban ettől eltérő portot állított be, módosítsa a port=3306 értéket a megfelelőre.

A szerver sikeres elindulását a „Now listening on: https://localhost:7200” üzenet jelzi.

8.3 Frontend (React) indítása

A felhasználói felület futtatásához Node.js környezet szükséges.

Függőségek telepítése: Első alkalommal nyisson egy terminált a frontend mappájában, és futtassa a következő parancsot:

npm install

Ez letölti a működéshez szükséges összes könyvtárat a node_modules mappába.

Ezután pedig egymás után futtassa a következő parancsokat:

npm install react-router-dom

npm install axios

npm install sweetalert2

Alkalmazás indítása: A telepítés végeztével indítsa el a fejlesztői szervert:

npm run dev

A terminálban megjelenő linkre (alapértelmezetten http://localhost:5173) kattintva a böngészőben megnyílik a Jogsifoglaló alkalmazás.

9. Nehézségek és megoldások

A fejlesztési folyamat során több olyan technikai és logikai akadályba ütköztünk, amelyek megoldása jelentős kutatómunkát igényelt.

CORS (Cross-Origin Resource Sharing) problémák: A fejlesztés kezdetén a Frontend és a Backend nem tudott kommunikálni a különböző portok miatt. Ezt a Program.cs fájlban beállított egyedi CORS-policy-val oldottuk meg, amely engedélyezte a hitelesítési fejlécek (Authorization Header) átvitelét is.

Többrétegű architektúra és DTO-k: Kihívást jelentett a rétegek közötti adatátvitel tiszta megvalósítása. Meg kellett tanulnunk, hogyan válasszuk szét az adatbázis-entitásokat a felhasználónak kiküldött DTO-któl, hogy ne fedjünk fel érzékeny adatokat (például jelszó hash-eket).

Aszinkronitás és Flow tesztek: A tesztelés során az aszinkron metódusok mockolása (Moq) és a ClaimsPrincipal szimulálása okozott fejtörést. Meg kellett oldanunk, hogy a tesztkörnyezet elhiggye a kontrollernek, hogy a kérést egy bejelentkezett Admin vagy Tanuló küldi.

Csapatmunka és integráció: Az utolsó fázisban a különböző modulok (pl. a profil törlése funkció) összehangolása igényelt extra erőforrást, hogy a Frontend minden esetben a megfelelő visszajelzést (SweetAlert2) adja a felhasználónak.

10. Továbbfejlesztési lehetőségek

Az alkalmazás jelenlegi formájában stabilan ellátja az alapfunkciókat, azonban a moduláris felépítésnek köszönhetően a jövőben több irányba is tovább bővíthető.

Valódi bankkártyás fizetés integrálása: Bár az adatbázis és a backend szinten a fizetési logika (tranzakciók tárolása, státuszkezelés) elkészült, a projekt következő lépcsőfoka egy valódi fizetési szolgáltató API-jának integrálása lenne. Ez lehetővé tenné a tanulók számára a biztonságos, azonnali online fizetést.

Automatikus értesítési rendszer: A felhasználói élmény fokozása érdekében egy olyan modul beépítése célszerű, amely a lefoglalt vezetési órák előtt (például 24 órával) automatikus értesítést (e-mail vagy SMS) küld a tanulónak és az oktatónak is, csökkentve ezzel az elfelejtett órák számát.

Naptár szinkronizáció: A Google Calendar API segítségével megvalósítható lenne, hogy a rendszerben rögzített időpontok azonnal megjelenjenek a résztvevők személyes naptárában is, így külső alkalmazásból is nyomon követhetővé válna az órarend.

Részletes oktatói profilok: Egy külön „Oktatóink" modul létrehozása, ahol a tanulók részletes információkat kaphatnának az oktatókról (szakmai tapasztalat, elérhetőségek, az oktatáshoz használt gépjármű típusa és specifikációi).

Vizuális testreszabhatóság (Sötét mód): A modern webes elvárásoknak megfelelően egy „Dark Mode" (sötét mód) opció beépítése a frontend oldalon, amely kényelmesebb használatot biztosít alacsony fényviszonyok mellett.

11. Összegzés

A „Jogsifoglaló” alkalmazás fejlesztése során egy olyan komplex, full-stack webes megoldást sikerült megvalósítanunk, amely valós piaci igényekre válaszolva digitalizálja az autósiskolák adminisztrációs folyamatait. A projekt nem csupán egy szoftverfejlesztési feladat volt, hanem egy átfogó mérnöki munka, amely a tervezéstől az adatbázis-architektúrán át a modern tesztelési módszertanokig minden lényeges fázist lefedett.

A fejlesztés során szerzett legfontosabb tapasztalatok:

Technológiai szinergia: Sikerült hatékonyan ötvözni a .NET Core Web API stabilitását és típusbiztonságát a React keretrendszer gyorsaságával és komponens alapú megközelítésével.

Biztonsági szemlélet: A JWT-alapú hitelesítés és a szerepkör alapú jogosultságkezelés implementálása során mélyebb betekintést nyertem az adatvédelem fontosságába és a modern webes biztonsági protokollok működésébe.

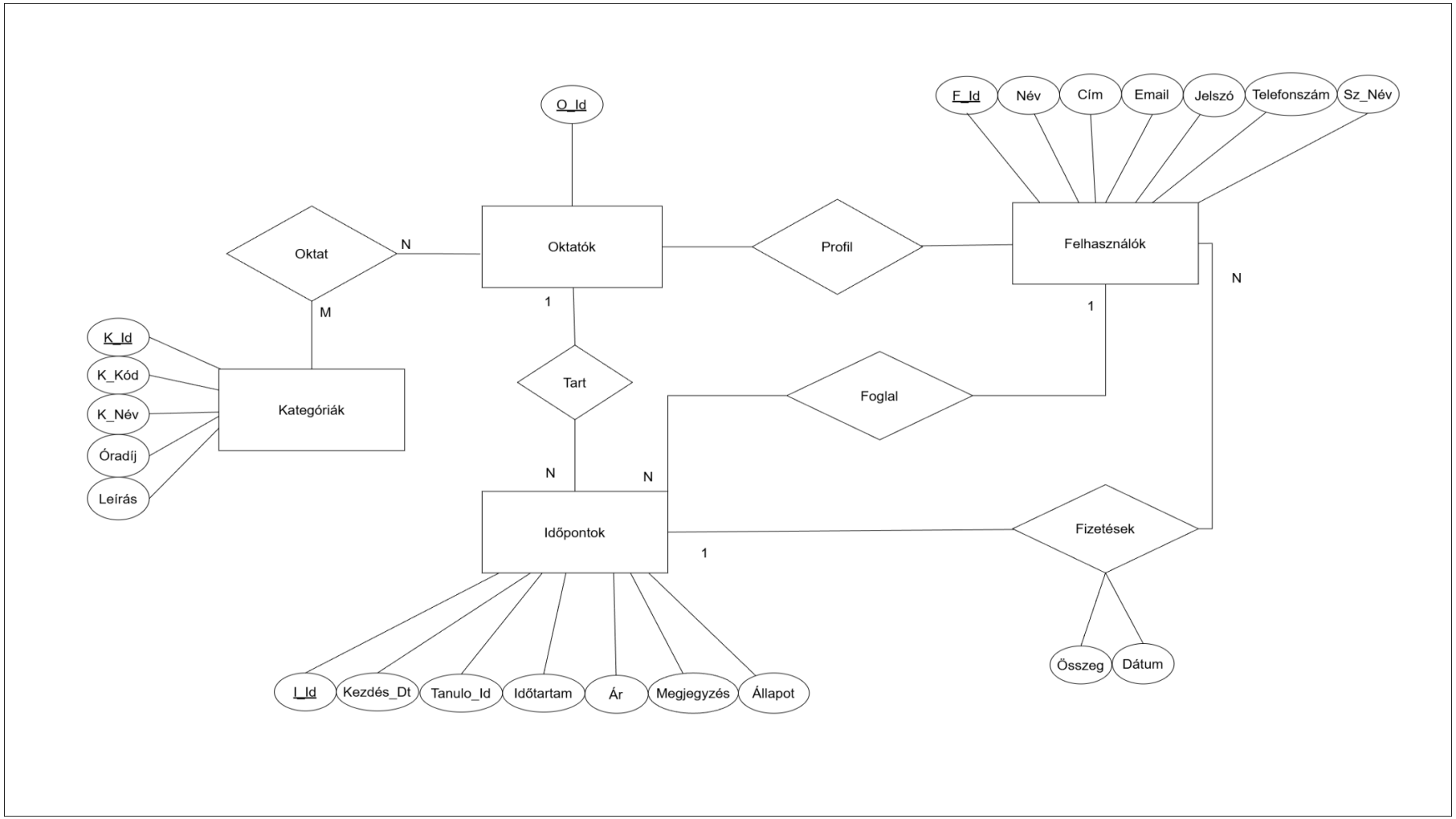
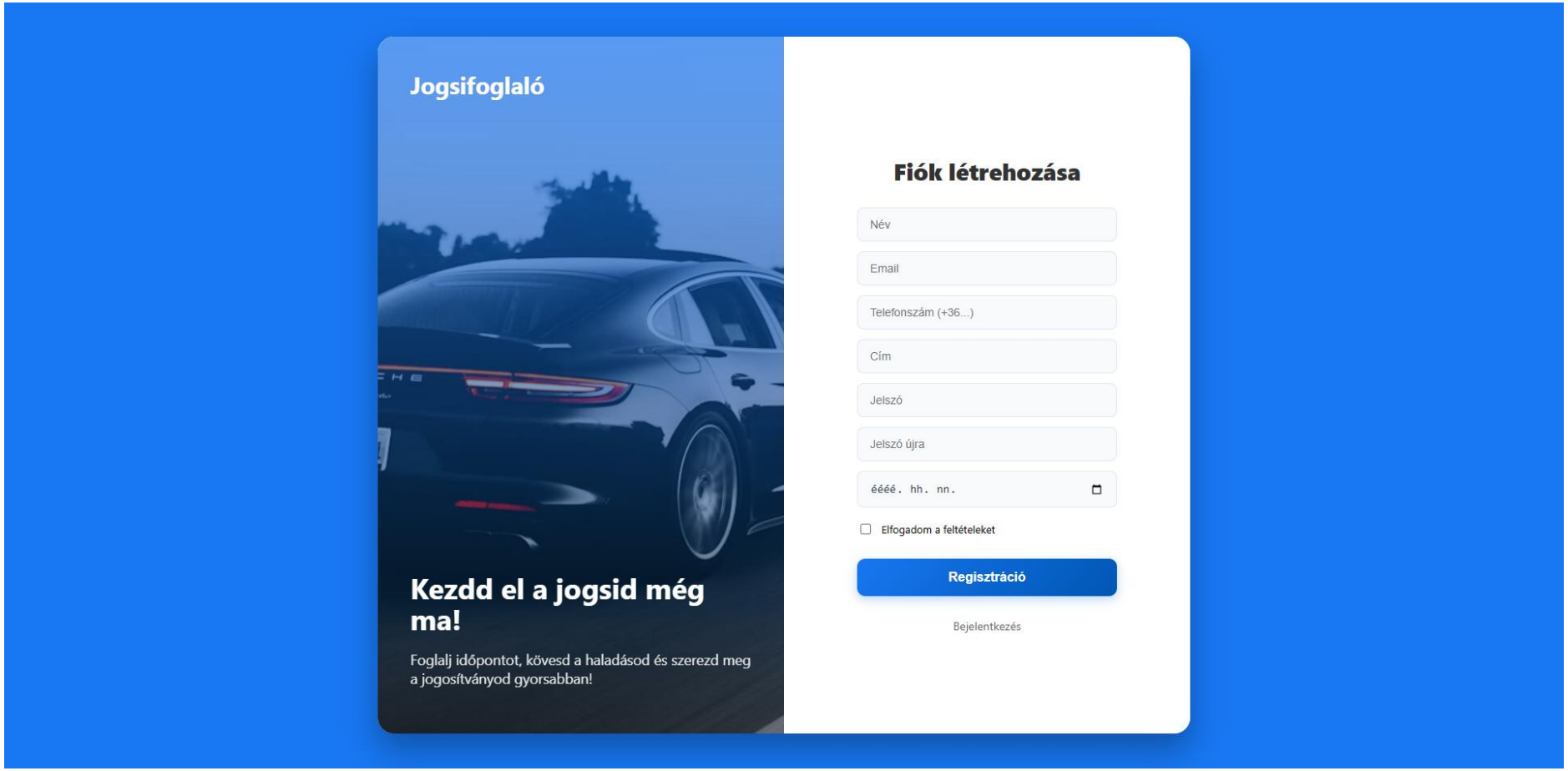
Minőségbiztosítás: A szoftverfejlesztő és -tesztelő szakmához hűen az automatizált xUnit egységtesztek és a komplex forgatókönyv alapú integrációs tesztek (Scenario Tests) garantálják az alkalmazás hibatűrését és hosszú távú fenntarthatóságát.

A projekt készítése közben felmerülő nehézségek – mint a CORS-konfigurációk vagy a rétegek közötti aszinkron adatfolyamok kezelése – fejlesztették a problémamegoldó készségemet és a technikai dokumentációk értelmezésének képességét.

Összességében elmondható, hogy a vizsgaremek teljesíti a specifikációban meghatározott követelményeket. Az alkalmazás egy skálázható, tiszta kódolási elveket követő szoftver, amely kiváló alapot nyújt a szakmai vizsgához, és a jövőben a tervezett továbbfejlesztési lehetőségekkel (például online fizetés, naptár szinkronizáció) akár éles piaci környezetben is megállhatja a helyét.

Mellékletek

ER-Modell



Regisztrációs oldal